Búsqueda Inteligente de Objetos en Simulación

Estado actual del proyecto y plan de implementación

Detección (YOLOE) + Búsqueda (VLMaps) + Navegación (Habitat-Sim)

Extensión: ROS 1 Noetic + Toyota HSR

Mario García Blasco

26 de marzo de 2026

Índice

1. Visión general del proyecto **3**
1.1. Problema . . . 3
1.2. Hipótesis . . . 3
1.3. Los tres niveles de conocimiento . . . 3
1.4. Idea clave: VLMaps estrecha la búsqueda, YOLOE confirma . . . 3
1.5. Etiquetado de habitaciones (LabelMe) . . . 4

2. Estado actual del proyecto (26 marzo 2026) **4**
2.1. Infraestructura — COMPLETADA . . . 4
2.2. Navegación interactiva — COMPLETADA . . . 4
2.3. Mapper LLM — COMPLETADO . . . 5
2.4. YOLOE como verificador — COMPLETADO . . . 5
2.5. Filtro de coherencia de heatmaps — COMPLETADO . . . 5
2.6. Filtro por tamaño medio de regiones — COMPLETADO . . . 5
2.7. Centrado visual con YOLOE — COMPLETADO . . . 6
2.8. Filtro de pared (wall filter) — COMPLETADO . . . 6
2.9. Room-based reasoning en Mapper LLM — COMPLETADO . . . 6
2.10. Colocación de objetos personalizados — COMPLETADO . . . 6
2.11. Lo que FALTA por implementar . . . 6

3. Arquitectura objetivo del sistema **7**
3.1. Flujo completo para una instrucción . . . 7
3.2. Flujo para objeto directo VLMap . . . 7

4. Los 5 métodos de búsqueda **7**
4.1. M1: Búsqueda aleatoria (baseline) . . . 8
4.2. M2: Candidato más cercano por BFS (implementado) . . . 8
4.3. M3: Búsqueda semántica (VLMap quality ranking) . . . 8
4.4. M4: Selección híbrida + navegación activa . . . 8
4.5. M5: Sistema completo . . . 8
4.6. Tabla resumen . . . 8

5. Componentes implementados en detalle **8**
5.1. Filtro de coherencia de heatmaps . . . 8
5.2. YOLOE en subproceso . . . 9
5.3. Scan multi-ángulo . . . 9
5.4. Detección y recuperación de colisiones . . . 9
5.5. Categorías directas, indirectas y YOLOE-incompatibles . . . 10
5.6. Filtro por tamaño medio de contornos . . . 10
5.7. Centrado visual (visual servoing) . . . 10
5.8. Filtro de pared (wall filter) . . . 10
5.9. Room-based reasoning . . . 11
5.10. Colocación de objetos personalizados . . . 11

6. Plan de implementación: trabajo futuro **11**
6.1. Paso 1 — Etiquetado de habitaciones (3 escenas) . . . 11
6.2. Paso 2 — Room-based reasoning en Mapper LLM . . . 12
6.3. Paso 3 — Heatmap region analysis completo . . . 12
6.4. Paso 4 — Implementar M1 y M3 baselines . . . 12
6.5. Paso 5 — Fórmula de scoring híbrida + M4 . . . 12
6.6. Paso 6 — M5: viewpoint planning + frontier exploration . . . 12
6.7. Paso 7 — Evaluación cuantitativa . . . 12
6.8. Paso 8 (Extensión) — Wrapper ROS 1 Noetic . . . 13

7. Orden de implementación **13**

8. Archivos clave del proyecto **13**

9. Tecnologías **13**

10. Preguntas frecuentes y decisiones de diseño **14**
10.1. P1: Categorías VLMap que YOLOE no reconoce (ej. “blinds”) . . . 14
10.2. P2: El path va por el otro lado de la pared . . . 14
10.3. P3: El robot navega demasiado cerca de paredes . . . 15
10.4. P4: El robot colisiona y se queda atascado . . . 15
10.5. P5: Cámara a –25° y scan multi-ángulo . . . 15
10.6. P6: Escenas MP3D más amplias . . . 16
10.7. P7: ¿Debería etiquetar las habitaciones? . . . 16
10.8. P8: ¿El robot usará la información de habitaciones? . . . 16
10.9. P9: ¿Puedo añadir objetos a la simulación para trials? . . . 17
10.10 P10: Visual centering (centrado del objeto en cámara) . . . 17
10.11 P11: Filtro de área media de contornos . . . 18

1. Visión general del proyecto

1.1. Problema

Dado un objetivo expresado en lenguaje natural (por ejemplo, *"find the laptop"*), un robot simulado debe localizar el objeto de forma eficiente en un entorno interior. El robot opera en escenas Matterport3D renderizadas con Habitat-Sim.

1.2. Hipótesis

Un sistema que combine tres niveles de conocimiento — etiquetas de habitaciones (LabelMe), memoria semántica de mobiliario (VLMaps, 37 categorías MP3D) y verificación visual de vocabulario abierto (YOLOE) — encuentra objetos específicos de forma más eficiente que métodos que dependen de un solo nivel.

1.3. Los tres niveles de conocimiento

Nivel	**Herramienta**	**Qué sabe**
Habitación	LabelMe (manual)	"Esto es el salón, esto el baño pequeño" — estructura espacial de alto nivel
Mueble / zona	VLMaps (37 cats MP3D)	"Hay una mesa aquí, un sofá allí" — localización de mobiliario genérico
Objeto específico	YOLOE (open-vocab)	"Hay un portátil encima de esa mesa" — verificación visual real

Cuadro 1: Los tres niveles de conocimiento del sistema.

1.4. Idea clave: VLMaps estrecha la búsqueda, YOLOE confirma

VLMaps conoce 37 categorías genéricas de MP3D (`table`, `chair`, `bed`, `sofa`, etc.) pero **no** puede detectar objetos específicos como un portátil, una botella o una pelota. Sin embargo, sabe *dónde están los muebles* sobre los que esos objetos suelen encontrarse.

YOLOE es un detector de vocabulario abierto: puede detectar **cualquier objeto** descrito con texto, a 161 FPS.

La combinación es:

1. El usuario pide un objeto específico ("laptop").
2. Un LLM traduce ese objeto a categorías VLMap donde es probable encontrarlo: `laptop` → `[table, counter, shelving]`.
3. VLMaps localiza todas las instancias de esas categorías en el mapa.
4. El robot navega al candidato más prometedor.
5. YOLOE mira con la cámara y confirma si el objeto está ahí.
6. Si no → siguiente candidato. Si sí → ENCONTRADO.

1.5. Etiquetado de habitaciones (LabelMe)

Para instrucciones del tipo *"go to the living room and find the laptop"*, el robot necesita saber dónde está el salón. Esto se resuelve con un etiquetado manual offline:

1. Se exporta el mapa top-down de la escena como imagen.
2. Se etiquetan las habitaciones con polígonos grandes en LabelMe.
3. Se convierte a un grid de etiquetas (`room_label_grid.npy`).
4. El robot consulta en qué habitación está cualquier punto del mapa.

Esto permite instrucciones compuestas: primero navegar a una habitación, luego buscar dentro.

2. Estado actual del proyecto (26 marzo 2026)

2.1. Infraestructura — COMPLETADA

- ✓ Docker con CUDA 12.1 + Habitat-Sim 0.3.1 + conda (Python 3.9)
- ✓ 3 escenas MP3D recolectadas con VLMaps construidos:
  - `gTV8FGcVJC9_1`, `jh4fc5c5qoQ_1`, `mJXqzFtmKg4_1`
- ✓ 4 escenas adicionales de **un solo nivel** descargadas (sin VLMap aún):
  - `mJXqzFtmKg4` (24 regiones), `jh4fc5c5qoQ` (16 regiones)
  - `JeFG25nYj2p` (22 regiones), `Uxmj2M2itWa` (31 regiones)
- ✓ Menú interactivo en Docker (`menu.sh`) para toda la pipeline
- ✓ Smart frame filtering (MIN_DIST=0.1m, MIN_ROT=5°) integrado en la recolección
- ✓ Submodule git `third_party/vlmaps` con commits separados

2.2. Navegación interactiva — COMPLETADA

- ✓ Pipeline completa: instrucción en lenguaje natural → LLM parsea → goal selection → visgraph → navegación live con visualización
- ✓ Goal selection con 3 niveles de fallback:
  1. Room target (LabelMe labels) → centroide de la habitación
  2. BFS sobre mapa erosionado → candidato VLMap más cercano por coste real
  3. Heatmap fallback → pico del heatmap semántico
- ✓ Visgraph con margen de 4px (∼20cm) de obstáculos (ampliado desde 2px)
- ✓ Navegación live (`execute_nav_live`) con detección de colisiones y hasta 3 replanes
- ✓ Recuperación de colisión: giro 30° + avance 2 pasos + replanificación
- ✓ Path visgraph mostrado como línea roja en el mapa top-down
- ✓ Heatmap semántico overlay (JET colormap, 50 % opacidad)

- ✓ Room overlay con etiquetas en centroides
- ✓ Cámara 1ª persona con inclinación de –25° (mira hacia abajo para ver mobiliario)
- ✓ Multi-goal retry (hasta 8 candidatos por categoría, ordenados por BFS)
- ✓ Detección de llegada a 1 metro (20 celdas a cs=0.05m)
- ✓ Approach + center: se acerca a 1m del objeto y rota para enfocarlo
- ✓ Lista de objetos navegables mostrada en cada prompt
- ✓ Try/except para no crashear si no encuentra un objeto

2.3. Mapper LLM — COMPLETADO

- ✓ El LLM traduce objetos no-VLMap a categorías VLMap probables
- ✓ Ejemplo: `laptop` → `[table, counter, shelving]`
- ✓ Distingue entre queries directos (sofa $\in$ MP3D) e indirectos (laptop $\notin$ MP3D)

2.4. YOLOE como verificador — COMPLETADO

- ✓ YOLOE integrado como verificador visual al llegar al candidato
- ✓ Ejecuta en subproceso separado (`yoloe_worker.py`) para evitar conflicto CUDA/OpenGL con Habitat-Sim
- ✓ Scan multi-ángulo: comprueba a 0°, –30° y +30° antes de descartar
- ✓ Categorías directas MP3D (sofa, bed...) confían en VLMap sin necesitar YOLOE
- ✓ Categorías indirectas (laptop → table) requieren confirmación YOLOE

2.5. Filtro de coherencia de heatmaps — COMPLETADO

- ✓ Filtro global para evitar falsos positivos en heatmaps VLMap
- ✓ Parámetros: mín. 30 celdas, $\geq$30 % dominancia, $\leq$20 componentes conexas
- ✓ Heatmap siempre se muestra (para debug), pero solo se usa para navegación si pasa el filtro
- ✓ Ejemplo: taburetes acolchados clasificados como "sofa" por CLIP (4058 celdas, 156 componentes) → rechazado por >20 componentes

2.6. Filtro por tamaño medio de regiones — COMPLETADO

- ✓ Filtro por región individual dentro de una categoría
- ✓ Calcula la media del área de todos los contornos de una categoría
- ✓ Descarta contornos cuya área cae por debajo del 30 % de la media
- ✓ Ejemplo: si 5 regiones de "sofa" tienen áreas 10, 133, 200, 100, 220 (media=132.6), la región de 10 celdas (7.5 % de la media) se descarta como falso positivo

2.7. Centrado visual con YOLOE — COMPLETADO

- ✓ Tras el approach basado en mapa, un bucle de servoing visual centra el objeto en la cámara
- ✓ Detecta el objeto con YOLOE, calcula el centro del bbox respecto al centro del frame
- ✓ Si el offset es >15 % del ancho del frame, gira 5° en la dirección correcta
- ✓ Hasta 6 iteraciones de corrección
- ✓ El resultado de la detección se reutiliza para la verificación (no duplica inferencias YOLOE)

2.8. Filtro de pared (wall filter) — COMPLETADO

- ✓ Detecta candidatos al otro lado de una pared comparando coste BFS vs distancia euclídea
- ✓ Si BFS/euclídea > 3×, el candidato se descarta (el path probablemente rodea una pared)
- ✓ Se aplica en `find_all_navigable_object_goals()` antes de ordenar candidatos

2.9. Room-based reasoning en Mapper LLM — COMPLETADO

- ✓ El Mapper LLM ahora sugiere habitaciones probables además de muebles
- ✓ Ejemplo: `laptop` → rooms: `[office, bedroom, living_room]`, cats: `[table, counter, shelving]`
- ✓ Cuando hay etiquetas de habitación disponibles, el robot navega primero a la habitación sugerida
- ✓ Prompt actualizado con 6 few-shot examples que incluyen rooms

2.10. Colocación de objetos personalizados — COMPLETADO

- ✓ Script `place_objects.py` para definir y colocar objetos 3D (.glb) en la escena
- ✓ Auto-carga: `interactive_object_nav.py` busca `placements.json` en el directorio de datos de la escena
- ✓ Los objetos son visibles para YOLOE pero NO aparecen en VLMap (por diseño)
- ✓ Config JSON con nombre, modelo .glb, posición (x,y,z) y escala

2.11. Lo que FALTA por implementar

- ∼ Etiquetado de habitaciones de las escenas (pipeline existe, falta etiquetar)
- ∼ Construir VLMaps para nuevas escenas (JeFG25nYj2p, Uxmj2M2itWa)
- × Heatmap region analysis completo (fórmula de calidad: área, densidad, intensidad)
- × Fórmula de scoring híbrida ($\alpha \cdot visual + \beta \cdot semantic - \gamma \cdot cost$)
- × M1 baseline: navegación aleatoria

- × M3 baseline: ranking por calidad semántica VLMap
- × M4–M5: navegación activa (look-while-move, viewpoints, frontiers)
- × Evaluación cuantitativa automatizada
- × Wrapper ROS 1 Noetic para Toyota HSR

3. Arquitectura objetivo del sistema

3.1. Flujo completo para una instrucción

Ejemplo: *"Go to the living room and find the laptop."*

1. **Parsing (LLM):** Extrae room=`living_room`, object=`laptop`.
2. **Navegación a habitación (LabelMe):** Consulta el grid de etiquetas, obtiene el centroide del salón, navega con visgraph.
3. **Mapper semántico (LLM):** `laptop` no está en las 37 categorías VLMap. El LLM traduce: `laptop` → `[table, counter, shelving]`. Estas son las categorías VLMap donde es probable encontrar un portátil.
4. **Búsqueda local (VLMap):** Genera heatmaps para `table`, `counter`, `shelving` **dentro del salón**. Identifica candidatos (mesas concretas). Ordena por score o proximidad según el método activo (M1–M5).
5. **Navegación al candidato:** Visgraph planifica ruta hasta la mesa con mejor score. El robot navega mostrando la línea roja. Si colisiona, se recupera (giro + replanificación).
6. **Approach + visual centering:** Al llegar a 1m del mueble candidato, el robot se acerca, rota para enfocarlo, y un bucle de servoing visual ajusta el heading hasta que el objeto queda centrado en la cámara ($\pm 15\,\%$ del centro del frame).
7. **Verificación (YOLOE):** El centrado visual ya ejecuta YOLOE. Si no detecta de frente, escanea a $\pm 30°$. Si detecta con confianza > umbral → **ENCONTRADO**. Si no → siguiente candidato.
8. **Iteración:** Hasta 8 candidatos por categoría. Si ninguno contiene el laptop, la búsqueda se agota.

3.2. Flujo para objeto directo VLMap

Ejemplo: *"Find the sofa."*

1. **Parsing:** object=`sofa` (está en las 37 categorías).
2. **VLMap directo:** Heatmap de `sofa`, hasta 8 candidatos por BFS.
3. **Verificación:** VLMap confirma proximidad (sin YOLOE). Las categorías MP3D directas confían en el mapa semántico.

4. Los 5 métodos de búsqueda

Todos comparten la misma infraestructura de navegación (`execute_nav_live`, visgraph, visualización). Lo que cambia es la **estrategia de selección de goal** y el **comportamiento durante el tránsito**.

4.1. M1: Búsqueda aleatoria (baseline)

- **Goal selection:** Punto aleatorio navegable del mapa.
- **Durante tránsito:** Nada.
- **Verificación:** YOLOE al llegar.
- **Si no encuentra:** Otro punto aleatorio.
- **Estado:** No implementado.

4.2. M2: Candidato más cercano por BFS (implementado)

- **Goal selection:** Si el objeto está en VLMap, BFS al más cercano. Si no, mapper LLM → categorías relacionadas → BFS al mueble más cercano.
- **Durante tránsito:** Nada (navegación directa con detección de colisiones).
- **Verificación:** YOLOE al llegar (scan multi-ángulo 0°, ±30°) para indirectos. VLMap para directos.
- **Si no encuentra:** Siguiente candidato por distancia (hasta 8 candidatos).
- **Estado:** Implementado y funcional.

4.3. M3: Búsqueda semántica (VLMap quality ranking)

- **Goal selection:** Candidatos ordenados por heatmap region quality (fórmula: $0{,}3 \cdot \text{area} + 0{,}4 \cdot \text{mean_score} + 0{,}3 \cdot \text{density}$).
- **Verificación:** YOLOE al llegar.
- **Estado:** No implementado.

4.4. M4: Selección híbrida + navegación activa

- **Goal selection:** Fórmula híbrida: $\text{score}(c) = \alpha \cdot \text{visual}(c) + \beta \cdot \text{semantic}(c) - \gamma \cdot \text{cost}(c)$.
- **Durante tránsito:** Look-while-move + re-routing.
- **Estado:** No implementado.

4.5. M5: Sistema completo

- **Goal selection:** Igual que M4 + viewpoint planning (4 poses alrededor) + frontier exploration.
- **Estado:** No implementado.

4.6. Tabla resumen

5. Componentes implementados en detalle

5.1. Filtro de coherencia de heatmaps

Cuando VLMaps genera un heatmap para una categoría, el resultado puede contener falsos positivos: embeddings CLIP similares que no corresponden al objeto real. Por ejemplo, taburetes acolchados clasificados como "sofa".

El filtro rechaza heatmaps incoherentes con tres criterios:

Método	Goal sel.	Look-move	Re-route	Viewpoints	Frontiers	Estado
M1: Aleatorio	Random	No	No	No	No	Pendiente
M2: Cercano	BFS dist	No	No	No	No	Hecho
M3: Semántico	VLMap quality	No	No	No	No	Pendiente
M4: Híbrido	$\alpha v + \beta s - \gamma c$	Sí	Sí	Frontal	No	Pendiente
M5: Completo	$\alpha v + \beta s - \gamma c$	Sí	Sí	4 ángulos	Sí	Pendiente

Cuadro 2: Comparación de los 5 métodos. M2 es el único completamente implementado.

- **Mín. 30 celdas totales:** Evita ruido de píxeles sueltos.
- **≥30 % dominancia:** La categoría ganadora debe dominar al menos el 30 % de las celdas activas.
- **≤20 componentes conexas:** Un heatmap muy fragmentado indica ruido CLIP, no un objeto real.

Ejemplo: taburetes acolchados generaban un heatmap "sofa" con 4058 celdas pero 156 componentes conexas. El filtro lo rechaza ($156 > 20$).

5.2. YOLOE en subproceso

El detector YOLOE utiliza MobileCLIP como text encoder, que inicializa CUDA internamente. Esto entra en conflicto con el contexto OpenGL de Habitat-Sim, causando un segfault.

La solución es ejecutar YOLOE en un subproceso completamente separado:

1. El proceso principal guarda el frame de cámara como imagen temporal (.jpg).
2. Lanza `yoloe_worker.py` como subproceso con los argumentos: imagen, nombre del objeto, umbral de confianza, ruta de pesos.
3. El worker carga YOLOE, ejecuta la inferencia y devuelve las detecciones como JSON por stdout.
4. El proceso principal parsea el JSON y toma la decisión.

5.3. Scan multi-ángulo

Al llegar al candidato, el robot no solo mira de frente: rota ±30° y ejecuta YOLOE desde 3 ángulos. Esto reduce falsos negativos por:

- Objetos ligeramente fuera del campo visual.
- Error acumulado en la rotación de approach.
- Oclusión parcial desde un solo ángulo.

5.4. Detección y recuperación de colisiones

El loop de navegación monitoriza `robot.sim.previous_step_collided` después de cada acción. Si detecta una colisión:

1. Gira 30° a la derecha.
2. Avanza 2 pasos.
3. Si sigue colisionando, gira otros 30°.
4. Replanifica la ruta desde la posición actual.
5. Se permiten hasta 3 replanificaciones antes de abandonar.

5.5. Categorías directas, indirectas y YOLOE-incompatibles

El sistema clasifica cada query en tres tipos:

- **Directas:** El objeto pertenece a las 37 categorías MP3D (e.g., `sofa`, `bed`, `table`). VLMap lo localiza directamente. YOLOE verifica visualmente al llegar — igual que para indirectas.
- **Indirectas:** El objeto no está en MP3D (e.g., `laptop`, `remote`). El Mapper LLM traduce a categorías VLMap probables. YOLOE confirma visualmente la presencia del objeto real.
- **YOLOE-incompatibles:** 15 categorías MP3D arquitectónicas o abstractas que YOLOE no puede detectar fiablemente: `void`, `wall`, `floor`, `ceiling`, `stairs`, `column`, `beam`, `railing`, `lighting`, `window`, `door`, `objects`, `furniture`, `appliances`, `board_panel`, `clothes`. Para estas, VLMap confirma por proximidad sin YOLOE.

5.6. Filtro por tamaño medio de contornos

Dentro de una categoría VLMap (e.g., "sofa"), hay varios contornos correspondientes a objetos detectados. Algunos son falsos positivos (blobs pequeños de ruido CLIP). El filtro calcula la media del área de todos los contornos y descarta los que caen por debajo del 30 % de esa media.

Ejemplo con 5 contornos de "sofa": áreas = [10, 133, 200, 100, 220]. Media = 132.6. Umbral = 39.8. El contorno de 10 celdas se descarta como ruido.

5.7. Centrado visual (visual servoing)

Tras la aproximación basada en mapa (`approach_and_center_object`), un bucle de feedback visual refina el heading del robot:

1. YOLOE detecta el objeto en el frame actual.
2. Calcula el centro X del bounding box respecto al centro del frame (640×480).
3. Si el offset es >15 % del ancho del frame, gira 5° en la dirección de corrección.
4. Repite hasta 6 iteraciones o hasta que el objeto esté centrado.

Esto reemplaza la dependencia de coordenadas de mapa (que acumulan error) por verificación visual directa.

5.8. Filtro de pared (wall filter)

El visgraph puede encontrar rutas que rodean paredes, enviando al robot a un candidato que está técnicamente alcanzable pero en la habitación equivocada. El filtro compara el coste BFS (distancia real caminando) con la distancia euclídea (línea recta):

$$\text{ratio} = \frac{\text{BFS cost}}{\text{distancia euclídea}}$$

Si ratio > 3,0 y la distancia euclídea es > 5 celdas, el candidato se descarta: el path probablemente rodea una pared. Esto es conservador — ratio = 1,4 es normal (caminos no son rectos), pero ratio > 3 indica un desvío por obstáculo grande.

5.9. Room-based reasoning

Cuando el usuario busca un objeto indirecto (e.g., "laptop"), el Mapper LLM sugiere no solo muebles (`[table, counter]`) sino también habitaciones (`[office, bedroom, living_room]`). Si las habitaciones están etiquetadas con LabelMe:

1. El robot navega primero a la habitación más probable (e.g., office).
2. Desde allí, busca los muebles sugeridos por BFS/heatmap.
3. Los candidatos cercanos a la habitación sugerida tienen prioridad natural (están más cerca después de la navegación a la habitación).

Sin etiquetas de habitación, el campo `rooms` se ignora y la búsqueda es global.

5.10. Colocación de objetos personalizados

Para realizar trials con objetos específicos (laptop, botella, pelota), se usa un sistema de colocación basado en configuración JSON:

1. Crear `placements.json` en el directorio de datos de la escena.
2. Definir cada objeto con: nombre, modelo .glb, posición (x,y,z), escala.
3. Al iniciar `interactive_object_nav.py`, los objetos se colocan automáticamente en la escena usando la API de physics de Habitat-Sim.

Importante: Los objetos colocados son visibles para la cámara (YOLOE los detecta) pero **no aparecen en el VLMap** (fue construido antes de la colocación). Esto es por diseño: VLMap guía al mueble, YOLOE confirma el objeto encima.

Ejemplo de `placements.json`:

```
{
  "objects": [
    {"name": "laptop", "asset": "sphere.glb",
     "position": [2.1, 0.85, -1.3],
     "scale": [0.35, 0.02, 0.25]},
    {"name": "bottle", "asset": "sphere.glb",
     "position": [1.0, 0.9, -0.5],
     "scale": [0.04, 0.15, 0.04]}
  ]
}
```

Para modelos realistas, descargar .glb de YCB Object Set o Google Scanned Objects.

6. Plan de implementación: trabajo futuro

6.1. Paso 1 — Etiquetado de habitaciones (3 escenas)

Qué: Etiquetar las 3 escenas con LabelMe. Polígonos grandes cubriendo habitaciones completas.

Cómo: Usar el menú Docker (opción export + LabelMe + convert). Etiquetar: `living_room`, `bedroom`, `bathroom`, `kitchen`, `office`, etc.

Impacto: Habilita instrucciones compuestas ("go to the living room and find the laptop") y room-based reasoning en el Mapper LLM.

6.2. Paso 2 — Room-based reasoning en Mapper LLM

Qué: Extender el LLM para que sugiera no solo categorías de muebles, sino también habitaciones probables.

Ejemplo: `laptop` → habitaciones: `[office, bedroom, livingroom]`; muebles: `[table, counter, shelving]`.

Beneficio: El robot busca primero en las habitaciones más probables en vez de explorar todo el mapa.

6.3. Paso 3 — Heatmap region analysis completo

Qué: No usar el heatmap VLMap como píxeles crudos, sino extraer regiones estructuradas con calidad.

Fórmula: $\text{quality}(r) = 0{,}3 \cdot \text{norm}(\text{area}) + 0{,}4 \cdot \text{mean_score} + 0{,}3 \cdot \text{density}$

Impacto: Base para M3 (ranking por calidad semántica).

6.4. Paso 4 — Implementar M1 y M3 baselines

M1 (aleatorio): Goal = punto aleatorio navegable. Sirve como baseline mínimo.

M3 (semántico): Goal = región con mayor quality score. Mide si el score semántico es mejor que la proximidad pura (M2).

6.5. Paso 5 — Fórmula de scoring híbrida + M4

Fórmula: $\text{score}(c) = \alpha \cdot \text{visual}(c) + \beta \cdot \text{semantic}(c) - \gamma \cdot \text{cost}(c)$

M4: Añade look-while-move (YOLOE durante tránsito si pasa cerca de hotspot VLMap) y re-routing (cambio de destino si detecta candidato mejor).

6.6. Paso 6 — M5: viewpoint planning + frontier exploration

Viewpoints: Al llegar a un candidato, el robot se coloca en 4 puntos alrededor (a 1.5m, cada 90°) y ejecuta YOLOE desde cada uno. Reduce falsos negativos por oclusión.

Frontiers: Si se agotan todos los candidatos, explorar zonas inexploradas priorizando las cercanas a regiones VLMap relevantes.

6.7. Paso 7 — Evaluación cuantitativa

Métricas primarias:

- **Success Rate:** Fracción de episodios donde el objeto se encuentra.
- **Correct First Goal:** Fracción donde el primer candidato visitado es correcto.
- **Path Length (SPL):** Distancia total recorrida.
- **Time to Find:** Pasos de simulación hasta encontrar.
- **Wrong Visits:** Candidatos visitados antes del correcto.

Output: CSV con resultados + gráficas comparativas para la memoria del TFG.

6.8. Paso 8 (Extensión) — Wrapper ROS 1 Noetic

Qué: Paquete ROS que conecta el sistema con el Toyota HSR.
Componentes:

- `vlmap_server` node: carga VLMap, expone servicio `/vlmap/query`.
- `vlmap_nav` node: recibe instrucciones, parsea con LLM, envía goals a `move_base`.
- Visualización en RViz: heatmap como `OccupancyGrid`, path como `nav_msgs/Path`.

7. Orden de implementación

Prioridad	Paso	Depende de	Estado
—	Mapper LLM (objeto → cats VLMap)	—	Hecho
—	YOLOE como verificador (subproceso)	—	Hecho
—	M2 baseline (BFS + multi-goal + collision)	—	Hecho
—	Filtro de coherencia de heatmaps	—	Hecho
—	Categorías directas vs indirectas	—	Hecho
—	Filtro de pared (ratio BFS/euclídea $> 3\times$)	—	Hecho
—	Filtro de área media (contornos $< 30\,\%$ media)	—	Hecho
—	Visual centering (servo YOLOE)	—	Hecho
—	Room-based reasoning LLM	—	Hecho
—	Custom object placement utility	—	Hecho
—	Descarga escenas single-level	—	Hecho
—	Categorías YOLOE-incompatible (15 cats)	—	Hecho
1	Etiquetado habitaciones (4 escenas)	—	Pipeline lista
2	Build VLMaps nuevas escenas	Escenas descargadas	Pendiente
3	Heatmap region analysis	—	Pendiente
4	M1 + M3 baselines	Paso 3	Pendiente
5	Fórmula híbrida + M4	Paso 3	Pendiente
6	M5 (viewpoints + frontiers)	Paso 5	Pendiente
7	Evaluación cuantitativa	Pasos 4–6	Pendiente
8	Wrapper ROS Noetic	—	Pendiente

Cuadro 3: Estado de implementación y próximos pasos.

8. Archivos clave del proyecto

9. Tecnologías

- **Simulador:** Habitat-Sim 0.3.1 + Matterport3D
- **Mapa semántico:** VLMaps (LSeg/CLIP, 37 categorías MP3D)
- **Detector:** YOLOE-11L-Seg (open-vocabulary, Ultralytics, en subproceso)
- **LLM:** GPT (parsing instrucciones + mapper semántico)
- **Path planning:** PyVisGraph + BFS sobre mapa erosionado
- **Room labels:** LabelMe (manual) → grid NumPy

Archivo	Rol
`application/interactive_object_nav.py`	Navegación interactiva principal. Loop de navegación, goal selection, YOLOE verification, collision recovery, visual centering, room reasoning, wall filter, mean-area filter.
`application/yoloe_worker.py`	Subproceso YOLOE (evita conflicto CUDA/OpenGL)
`application/place_objects.py`	Utilidad para colocar objetos 3D custom en escenas Habitat (JSON config)
`vlmaps/robot/habitat_lang_robot.py`	Robot + visgraph + mapa erosionado (margen 4px)
`vlmaps/utils/llm_utils.py`	Mapper LLM: objeto → categorías VLMap + habitaciones sugeridas
`vlmaps/utils/habitat_utils.py`	Sensores y configuración Habitat (cámara –25°)
`vlmaps/utils/room_map_utils.py`	Room labels, matching, persistencia
`docker/menu.sh`	Menú interactivo Docker

- **Infraestructura:** Docker + CUDA 12.1 + Python 3.9 + PyTorch
- **Extensión:** ROS 1 Noetic + Toyota HSR

10. Preguntas frecuentes y decisiones de diseño

10.1. P1: Categorías VLMap que YOLOE no reconoce (ej. "blinds")

Algunas de las 37 categorías MP3D son arquitectónicas o abstractas (como `wall`, `ceiling`, `beam`, `railing`, `blinds`) y YOLOE no las reconoce como objetos. Cuando el robot llegaba al objeto correcto y pedía confirmación a YOLOE, este no lo encontraba, y el robot seguía buscando inútilmente.

Solución implementada: Se define un conjunto de 15 **categorías YOLOE-incompatible**:

```
_YOLOE_INCOMPATIBLE = {
    "void", "wall", "floor", "ceiling",       # superficies
    "stairs", "column", "beam", "railing",     # arquitectura
    "lighting", "window", "door",              # fixtures
    "objects", "furniture", "appliances",       # demasiado genérico
    "board_panel", "clothes",                   # ambiguo
}
```

Solo estas 15 categorías saltan la verificación YOLOE y se dan por encontradas con `check_object_nearby()` del VLMap. **Todas las demás categorías** (incluyendo `chair`, `sofa`, `bed`, `table`, etc.) pasan por YOLOE, ya que la tesis evalúa el pipeline VLMaps + YOLOE completo.

10.2. P2: El path va por el otro lado de la pared

El visgraph planifica sobre el mapa de obstáculos erosionado, que es un mapa 2D. Si hay una pared fina entre el robot y el objetivo, el planificador puede encontrar un camino que rodea

la pared por el lado equivocado: técnicamente alcanzable, pero el robot acaba en la habitación incorrecta.

Solución implementada (doble):

1. **Filtro de pared:** En `find_all_navigable_object_goals()`, se calcula el ratio entre el coste BFS y la distancia euclídea al candidato. Si `ratio` $> 3\times$ (y la distancia euclídea $>$ 5 celdas), el candidato se descarta porque probablemente está al otro lado de una pared. Constante: `_WALL_RATIO_THRESH = 3.0`.

2. **Multi-goal retry:** Hasta 8 candidatos por categoría — si el primero falla, se intenta el siguiente.

10.3. P3: El robot navega demasiado cerca de paredes

El margen de erosión del mapa de navegación define la distancia mínima a obstáculos. Estaba configurado a 2px (∼10cm), insuficiente para un robot real.

Solución implementada: Se ha duplicado a **4px (∼20cm)**. Si fuera insuficiente, se puede subir a 6px (30cm) cambiando una sola variable: `_NAV_MARGIN_PX = 6` en `habitat_lang_robot.py`, línea 91.

10.4. P4: El robot colisiona y se queda atascado

Cuando el robot no encontraba un objeto, podía quedarse avanzando contra una pared indefinidamente.

Solución implementada: `execute_nav_live()` monitoriza `robot.sim.previous_step_collided` en cada paso. Si detecta colisión:

1. Gira 30° a la derecha.

2. Avanza 2 pasos.

3. Si sigue colisionando, gira otros 30°.

4. Replanifica la ruta desde la posición actual.

5. Se permiten hasta 3 replanificaciones antes de abandonar ese goal.

10.5. P5: Cámara a –25° y scan multi-ángulo

La cámara a –15° no capturaba bien los muebles bajos (mesas, sillas). Además, un solo ángulo frontal puede perder objetos ligeramente fuera del campo visual.

Solución implementada:

- Cámara inclinada a **–25°** (antes –15°), orientada hacia abajo para capturar mobiliario.

- Scan **multi-ángulo horizontal**: al llegar al candidato, YOLOE comprueba a 0°, gira 30° a la izquierda, comprueba, gira 30° a la derecha del centro, comprueba, y vuelve al centro. Tres ángulos que cubren ±30° de campo visual.

- No se cambia la inclinación vertical porque las acciones `look_up`/`look_down` no existen en el action space de Habitat configurado.

10.6. P6: Escenas MP3D más amplias

Se analizaron los archivos `.house` de múltiples escenas MP3D para encontrar las que cumplen: **un solo nivel** (sin escaleras entre pisos) y **espacios amplios** (pasillos anchos, habitaciones grandes). Se descartaron candidatos multi-nivel como `VzqfbhrpDEA` (2 niveles) y `1LXtFkjw3qL` (multi-piso).

4 escenas seleccionadas (todas de 1 nivel):

Escena	Regiones	Habitaciones
`mJXqzFtmKg4`	24	Gym, 5 hallways, 5 bathrooms, 3 bedrooms, dining, living, kitchen, office, laundry, closet, outdoor, entry
`JeFG25nYj2p`	22	5 hallways, 4 bedrooms, 3 bathrooms, 2 closets, toilet, office, lounge, living, kitchen, laundry
`jh4fc5c5qoQ`	16	3 porches, 2 living rooms, 2 kitchens, 2 laundry, 2 entries, 2 bedrooms, 2 bathrooms, office
`Uxmj2M2itWa`	31	Gran variedad de habitaciones (31 regiones), single-level

Estado de descarga: Las 4 escenas single-level están descargadas en `/home/mario/tfg/data/mp3d/`:

- `mJXqzFtmKg4` — 277MB, VLMap construido.
- `jh4fc5c5qoQ` — 136MB, VLMap construido.
- `JeFG25nYj2p` — 192MB, descargada (VLMap pendiente).
- `Uxmj2M2itWa` — 205MB, descargada (VLMap pendiente).

Alternativas verificadas como single-level: `8WUmhLawc2A` (18 regiones), `zsNo4HB9uLZ` (23 regiones).

10.7. P7: ¿Debería etiquetar las habitaciones?

Sí, absolutamente. La pipeline de etiquetado ya está lista (exportar mapa → LabelMe → convertir). Etiquetar las habitaciones permite:

- Instrucciones compuestas: "go to the kitchen and find the bottle".
- Room-based reasoning en el Mapper LLM: laptop → buscar primero en office/bedroom.
- Reducir drásticamente el espacio de búsqueda.

Importante: Dibujar polígonos **grandes** que cubran toda la habitación. Polígonos pequeños (como un "bedroom_main" de 99 píxeles) causan problemas porque sus centroides pueden caer sobre paredes u obstáculos.

10.8. P8: ¿El robot usará la información de habitaciones?

Implementado. El Mapper LLM ahora devuelve un campo `rooms"` además de las categorías VLMap. Ejemplo: `"laptop"` → `{categories: ["table", çounter"], rooms: [.ºffice", "bedroom"]}`.

Flujo:

1. El LLM sugiere habitaciones probables para el objeto pedido.
2. Si hay habitaciones etiquetadas en la escena, el robot navega primero al centroide de la habitación sugerida.

3. Desde ahí, inicia la búsqueda de muebles candidatos dentro de esa zona.
4. Si no se encuentra en la primera habitación, pasa a la siguiente sugerida.

También se pueden dar instrucciones directas tipo "go to the office and find the laptop" — el parser LLM ya separa room + object.

10.9. P9: ¿Puedo añadir objetos a la simulación para trials?

Implementado. Se ha creado la utilidad `application/place_objects.py` que permite colocar objetos 3D en escenas Habitat mediante un archivo JSON de configuración.

Uso:

```
# Generar config de ejemplo:
python place_objects.py --example --output placements.json

# El archivo placements.json se coloca en el directorio de datos
# de la escena y se carga automáticamente al iniciar la navegación.
```

Formato del JSON:

```
{ "objects": [
    { "name": "laptop", "asset": "sphere.glb",
      "position": [2.1, 0.85, -1.3],
      "scale": [0.35, 0.02, 0.25] }
  ] }
```

Carga automática: `interactive_object_nav.py` busca un archivo `placements.json` en el directorio de datos de la escena al iniciar. Si existe, coloca todos los objetos antes de empezar la navegación.

Detalle importante: Los objetos añadidos **no aparecen en el VLMap** (se construyó antes). El VLMap guía al mueble correcto (mesa), y YOLOE confirma visualmente que el objeto está encima. Esto es exactamente el pipeline que se quiere evaluar.

Se pueden usar modelos `.glb` del **YCB Object Set** o **Google Scanned Objects**, o usar `sphere.glb` de habitat-sim como stand-in con escalado para simular formas (elipsoide plano = laptop, cilindro alto = botella).

10.10. P10: Visual centering (centrado del objeto en cámara)

Al llegar a un candidato, el robot necesita centrar el objeto detectado en el campo visual para obtener una confirmación YOLOE fiable y una imagen clara.

Implementado: Función `_visual_center_object()` en `interactive_object_nav.py`. Funciona como un servo visual:

1. YOLOE detecta el objeto y devuelve bounding box `[x1, y1, x2, y2]`.
2. Se calcula el centro horizontal del bbox relativo al centro del frame.
3. Si el offset > 15 % del ancho del frame, el robot gira (turn_left o turn_right).
4. Se repite hasta que el objeto está centrado o se alcanzan 6 iteraciones.
5. Si YOLOE pierde el objeto durante el centrado, se devuelve el último resultado.

Parámetros: `max_iters = 6`, `tolerance_frac = 0.15` (15 % del ancho del frame).

10.11. P11: Filtro de área media de contornos

Al consultar el heatmap VLMap para una categoría (ej. "sofa"), se obtienen múltiples contornos. Algunos son ruido: manchas pequeñas que no corresponden a un objeto real.

Implementado: En `find_all_navigable_object_goals()`, se calcula la media de área de todos los contornos de la categoría. Los contornos con área $< 30\,\%$ de la media se descartan.

```
areas = [cv2.contourArea(c) for c in contours]
mean_area = sum(areas) / len(areas)
threshold = 0.30 * mean_area
kept = [c for c, a in zip(contours, areas) if a >= threshold]
```

Esto elimina artefactos pequeños y concentra la búsqueda en regiones significativas del mapa.